

# Outline

- Overview
- Domain specific languages

## Syntax trees

- Control flow diagrams
- Regular expressions

# Motivation

- Grammar refers to a set of rules that defines the structure and syntax of the language.
- Grammar in English: past perfect for the past in reported speech

After I **had finished** my presentation, I realized that I **had forgotten** to include an important detail.

- Grammar in programming languages:

**if + expression**

```
int number;  
if (number > 0) {  
    System.out.println("The number is positive!");  
}
```

**for (... ; ... ; ... )**

```
for (int i = 0; i < 10; i++) {  
    System.out.println("Number: " + i);  
}
```

- Adherence to grammatical rules trumps immediate meaning.

# Generation of the parse tree

```
<if_statement> ::= If <condition> Then <statement>
                  | If <condition> Then <statements> End If
                  | If <condition> Then <statements> Else <statements> End If

<condition> ::= <expression> <relational_operator> <expression>

<relational_operator> ::= < | > | <= | >= | =

<expression> ::= <expression> + <term> | <expression> - <term> | <term>

<term> ::= <term> * <factor> | <term> / <factor> | <factor>

<factor> ::= <identifier> | <number> | (<expression>) | <string_literal>

<statements> ::= <statement> <statements>

<statement> ::= <assignment> | <if_statement> | <for_statement> ...

<assignment> ::= <identifier> = <expression>

<identifier> ::= <letter> | <identifier><letter> | <identifier><digit>

<letter> ::= a | b | c | d | ... | A | B | C | D | ...

<digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<number> ::= <integer> | <integer>.<integer>

<integer> ::= <digit> | <integer><digit>

<string_literal> ::= <quote><character><quote> | <quote><characters><quote>

<character> ::= <letter> | <digit> | ...

<characters> ::= <character><characters>

<quote> ::= “
```

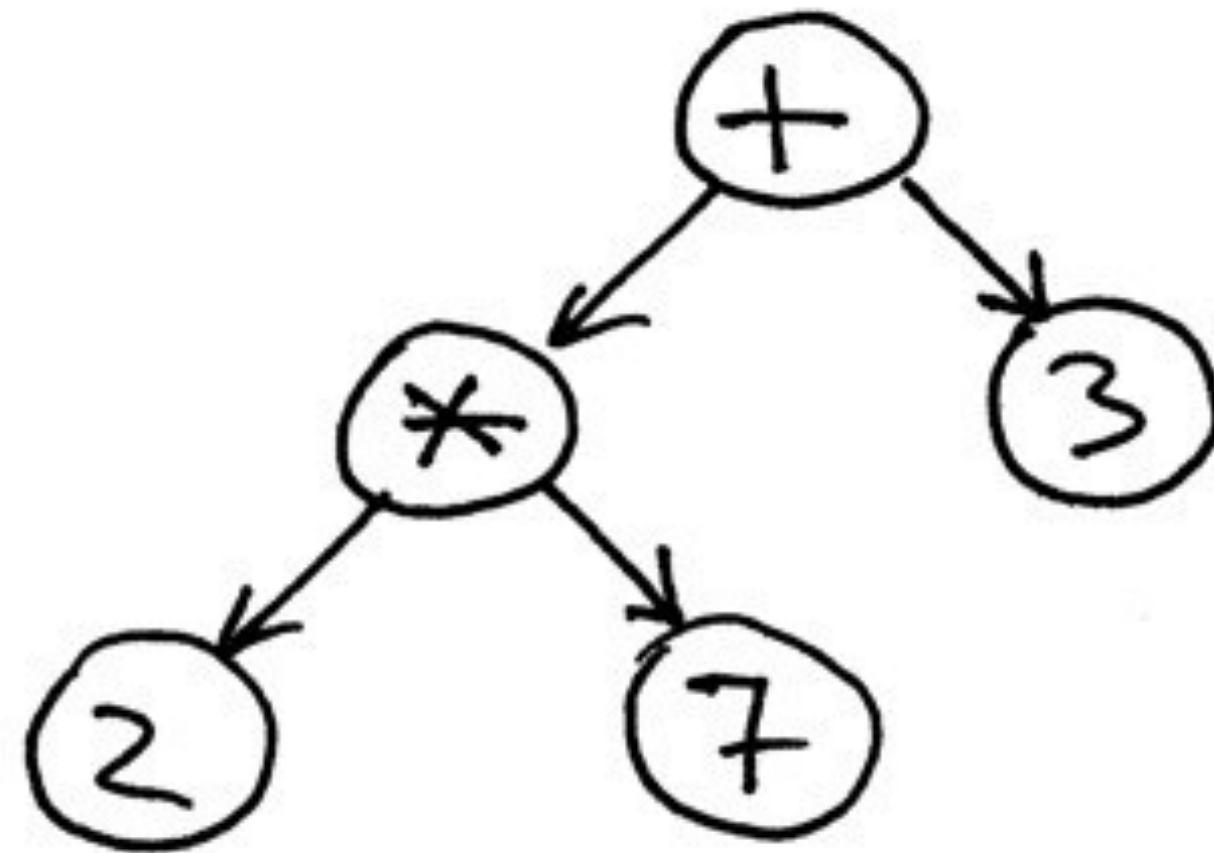
```
If Temperature <= 0 Then
    Description = "Freezing"
End If
```



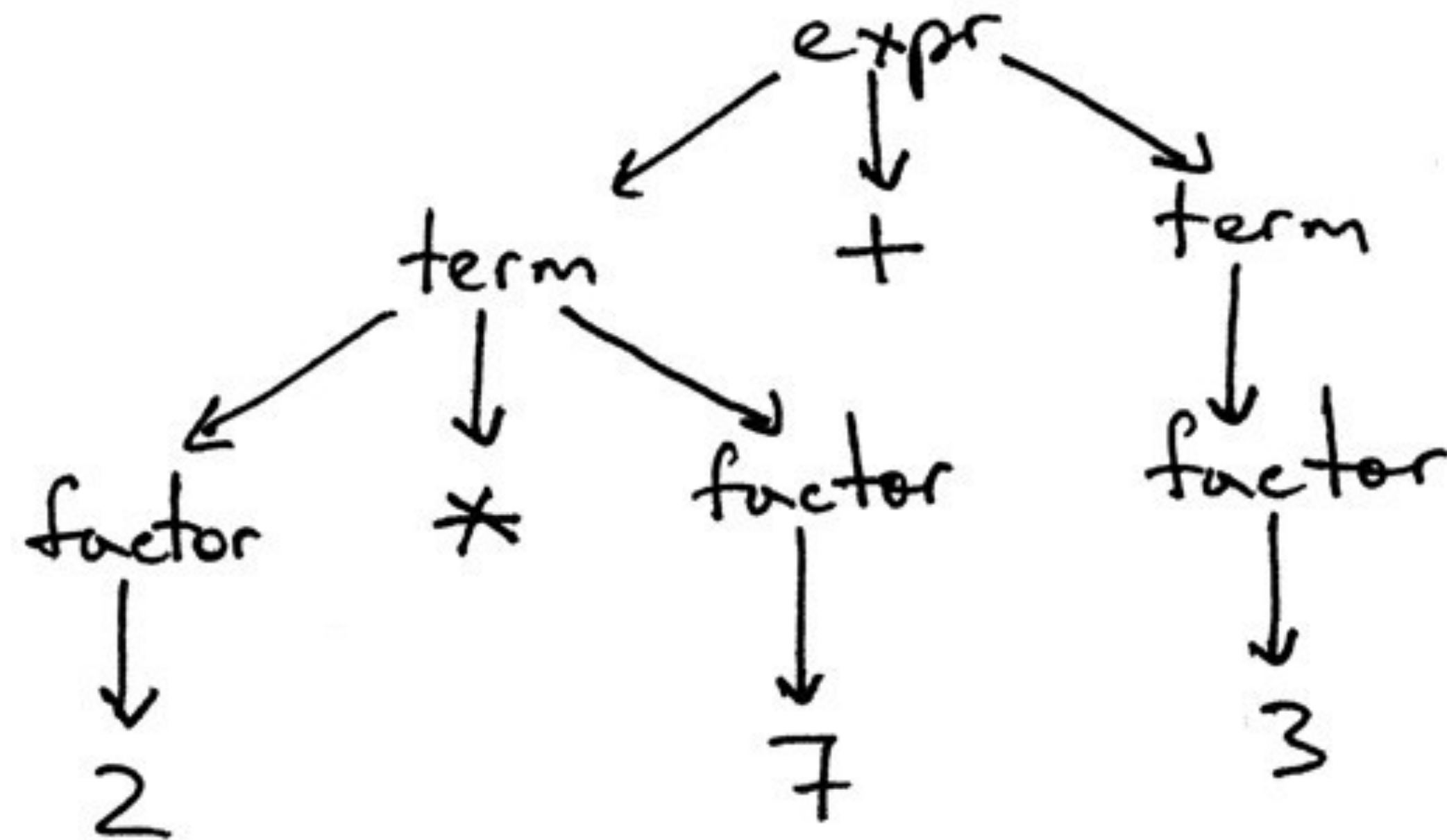
# Abstract syntax tree vs. parse tree

$2 * 7 + 3$

AST



Parse tree



# Syntax tree

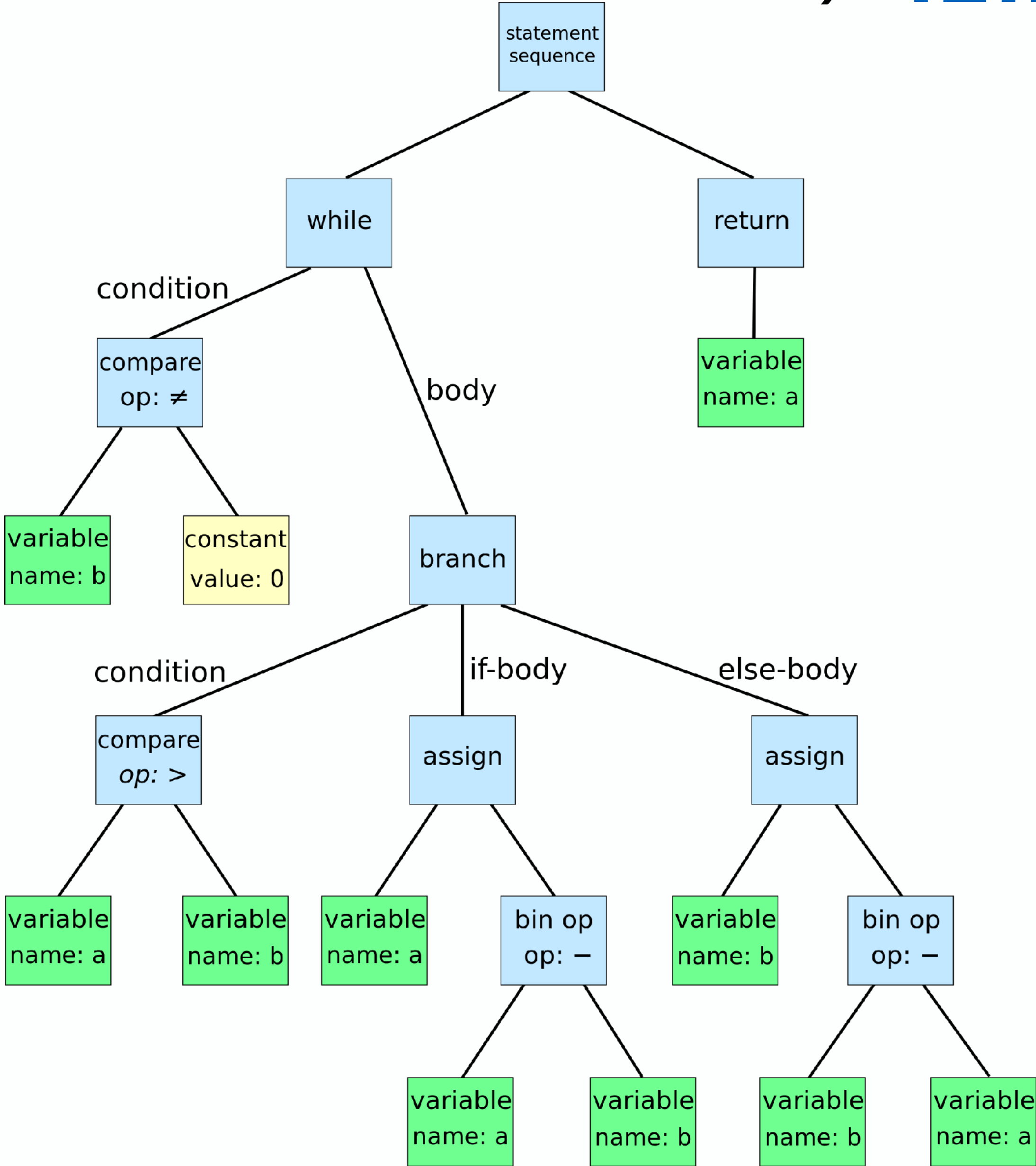


- **Tree representation** of the syntactic structure of text (often source code) written in a formal language
- Each node of the tree denotes a construct in the text
- The syntax is **abstract**: it does not represent every detail, but just the structural or content-related details
  - For **example**: grouping parentheses are implicit in the tree structure, so these do not have to be represented as separate nodes
  - A syntactic construct like an **if-else** statement may be denoted by means of a single node with e.g. three branches
- Used in **compilers**, **program analysis** and **program transformation systems**



# Example: Euclidean algorithm (greatest common divisor)

```
while b ≠ 0:
    if a > b:
        a := a - b
    else:
        b := b - a
return a
```



# Motivation for syntax trees



- Widely used in **compilers** to represent the **structure of program code**
- Usually the result of the **syntax analysis phase** of a compiler
- Serve as an **intermediate representation** of the program
- Have a **strong impact** on the final output of the compiler
- Properties
  - Can be edited and enhanced with information such as properties and annotations
  - Do not need to include inessential punctuation and delimiters
  - Contain extra information about the program
  - **Example**: store the position in the source code to print useful error messages

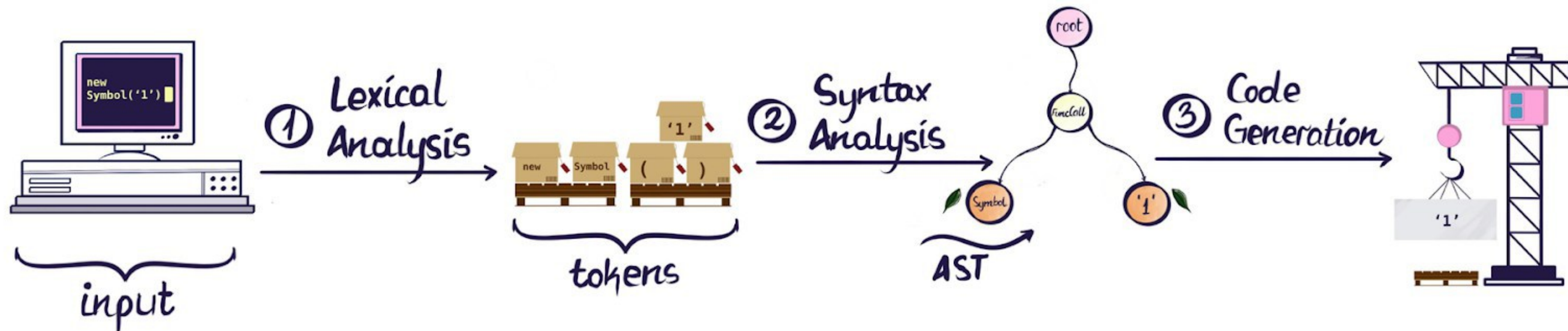
- The design of a syntax tree is often closely linked with the design of a **compiler** and its expected features
- Core requirements
  - Variable types and the location of each declaration in the source code must be preserved
  - The **order of executable statements** must be explicitly represented and well defined
  - Left and right components of binary operations must be stored and correctly identified
  - Identifiers and their assigned values must be stored for assignment statements



- **Semantic analysis:** the compiler checks for correct usage of the elements of the program and the language
  - Creation of symbol tables based on the **abstract syntax tree (AST)** during semantic analysis
- **Verification:** a complete traversal of the tree allows verification of the correctness of the program
- **Code generation:** the syntax tree serves as the base for code generation
  - Often used to generate an **intermediate representation**, sometimes called an intermediate language, for the code generation

# Compiler

- ASTs are tree representations of code
- Fundamental part of the way a compiler works
- When a compiler transforms some code there are fundamentally the following steps
  1. Lexical analysis
  2. Syntax analysis
  3. Code generation



# Integrated development environment (IDE)



- Many features of an IDE depend on ASTs, e.g.
    - Code formatting
    - Code highlighting
    - Code refactoring
  - **Round-trip capability**
    - Create the AST of the source code
    - Manipulate the AST (e.g. rename variables)
    - Write the changes back to the source code
- Easy to change code programmatically (using strongly typed objects in the AST)



# Example

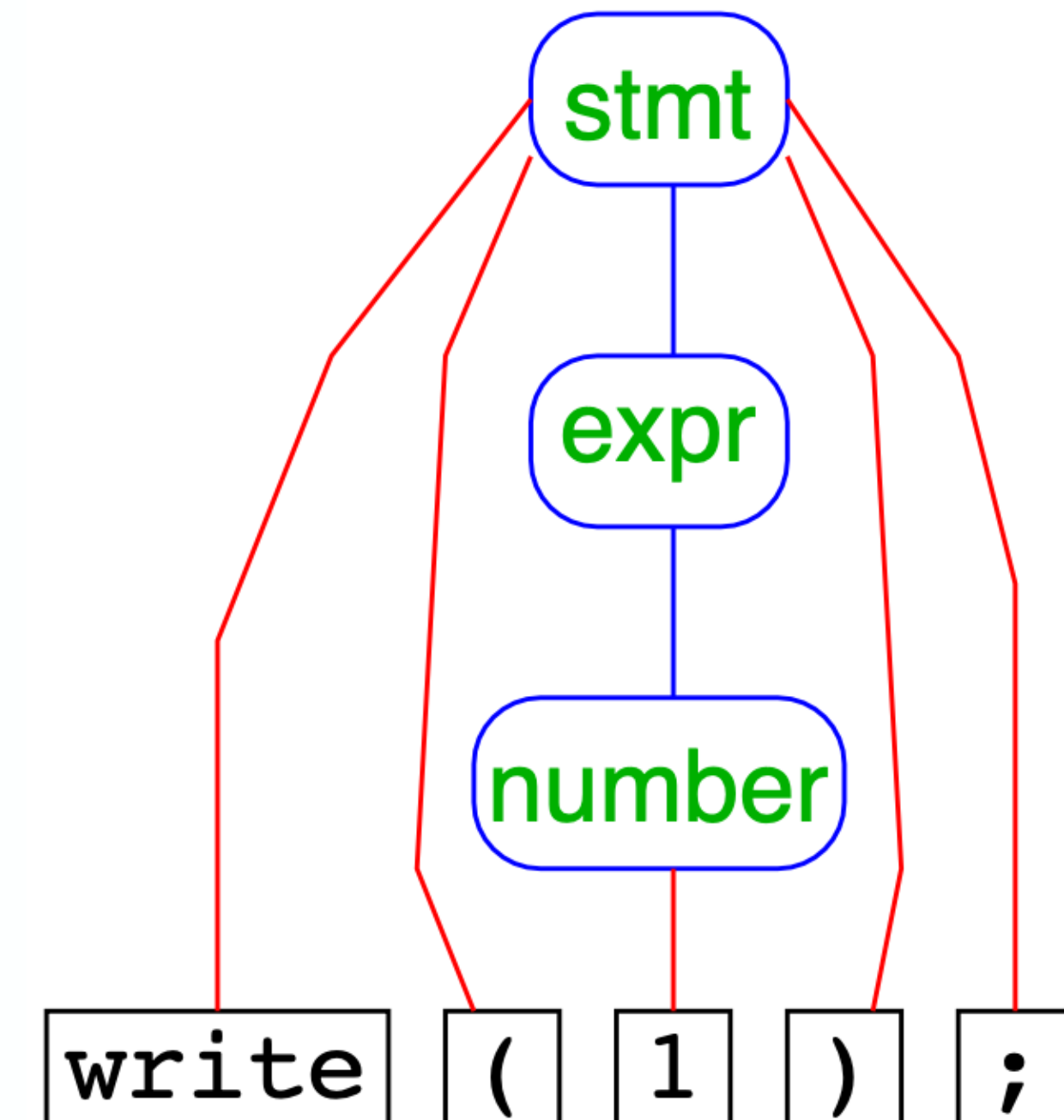
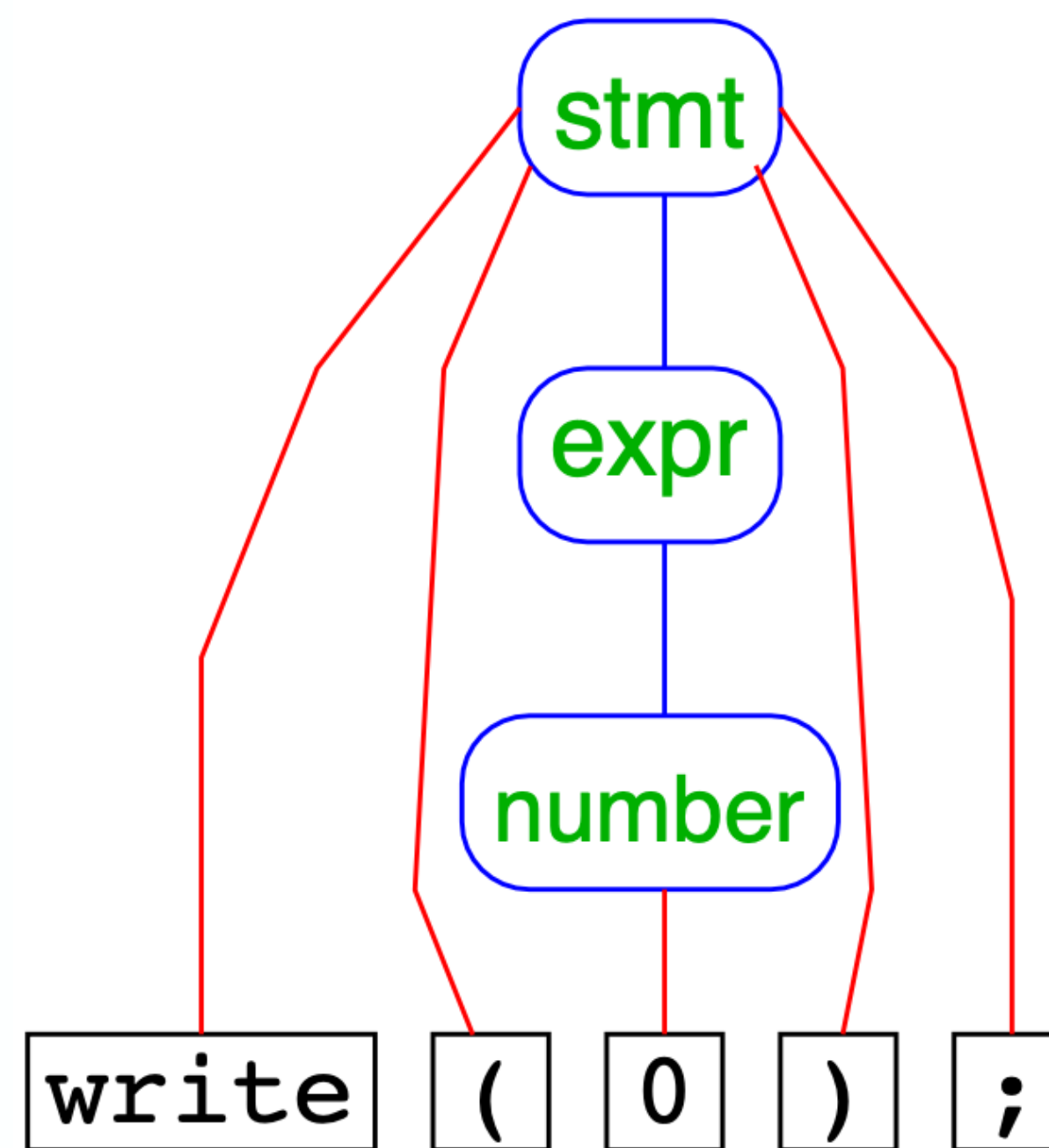
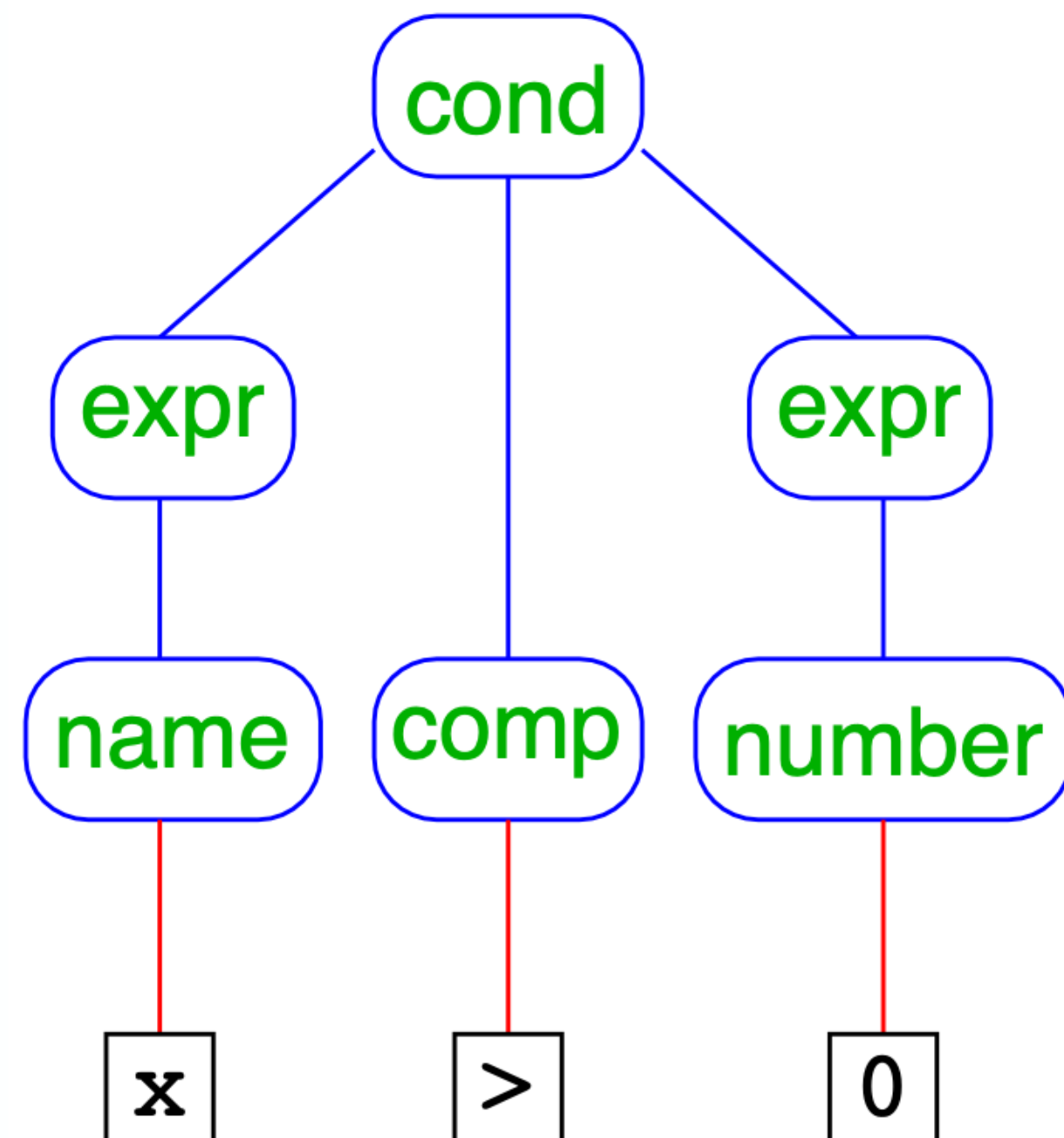
```
int x;  
x = readInt();  
if (x > 0) {  
    write(1);  
} else {  
    write(0);  
}
```

- The hierarchical structure of program fragments can be visualized by means of syntax trees
- **Note:** we ignore { and } in the syntax tree



# Syntax tree

- Syntax trees for **`x > 0`** and **`write(0);`** and **`write(1);`**



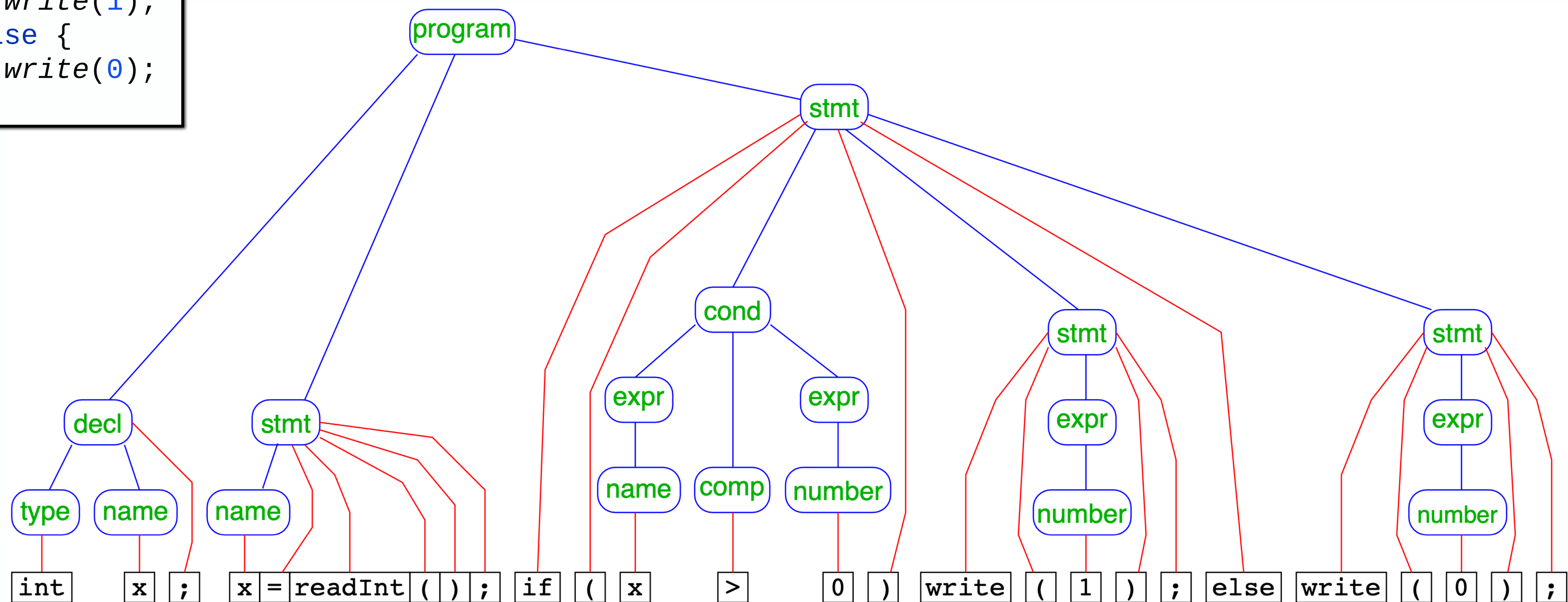
- Leaves: words / tokens
- Internal nodes: names of program fragments



# Syntax tree for the complete program

```
int x;  
x = readInt();  
if (x > 0) {  
    write(1);  
} else {  
    write(0);  
}
```

**Note:** we ignore { and } in the syntax tree



# Exercise W10E02

🕒 15 min



- Create a syntax tree for the following program
- Use Apollon on <https://apollon.ase.in.tum.de> to model it

```
int x = readInt();  
while (x > 0) {  
    write(x);  
    x = x - 1;  
}
```

- **Hint:** ignore `{` and `}` in the syntax tree